The frontend stores the access token and account summary in sessionStorage, limiting persistence to the current browser tab. This still exposes the token to any successful cross-site scripting attack. The Content Security Policy reduces some exposure, but production hardening should evaluate short-lived tokens, refresh rotation, and HttpOnly secure-cookie alternatives according to the deployed architecture.

API-driven pages do not replace failed responses with mock Job data. The current frontend uses server-side filters, pagination, and sorting for larger catalogues; skill/title/location/domain autocomplete; Candidate status tabs; CV review and confirmation; recruiter company onboarding; JD quality preview; Job draft/publish; urgent and deadline controls; Talent Pool bookmarks and invitations; per-account AutoFit settings; and analytics refetch for current-day data.

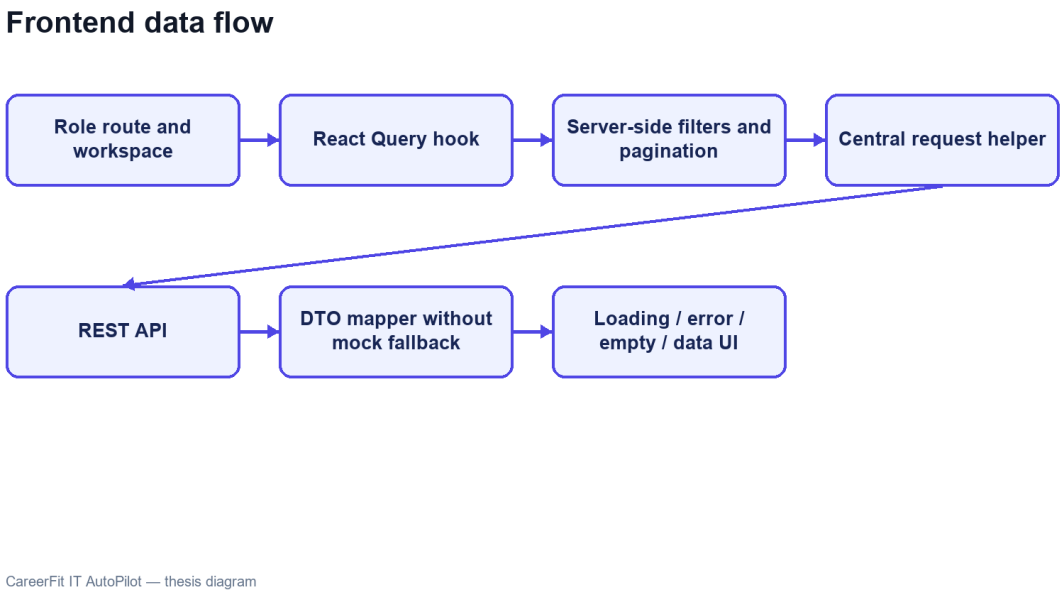
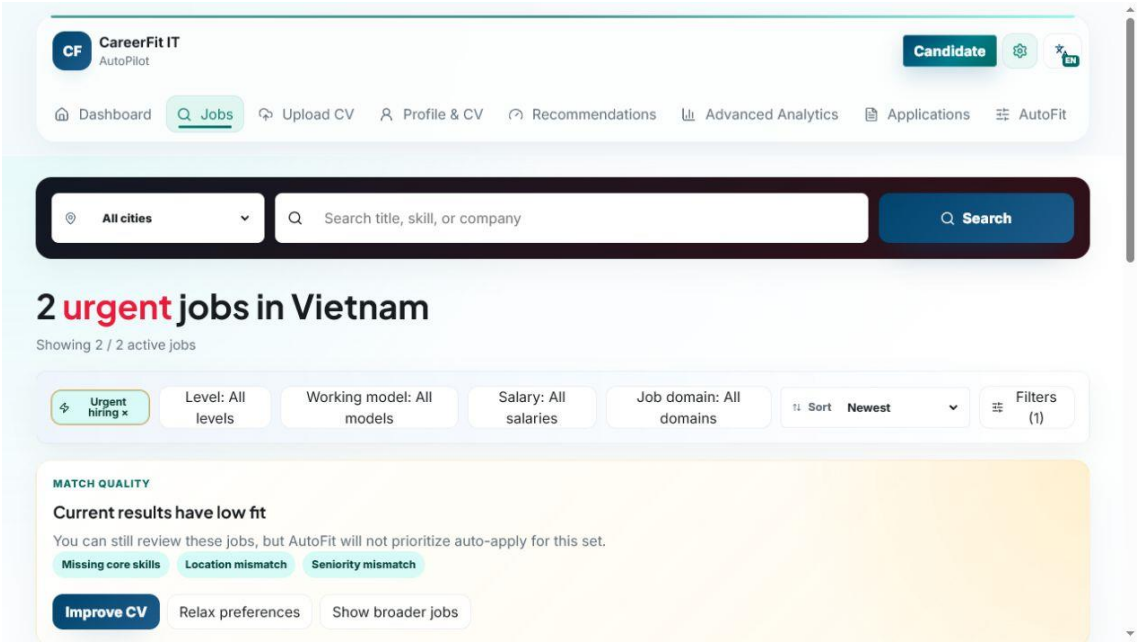
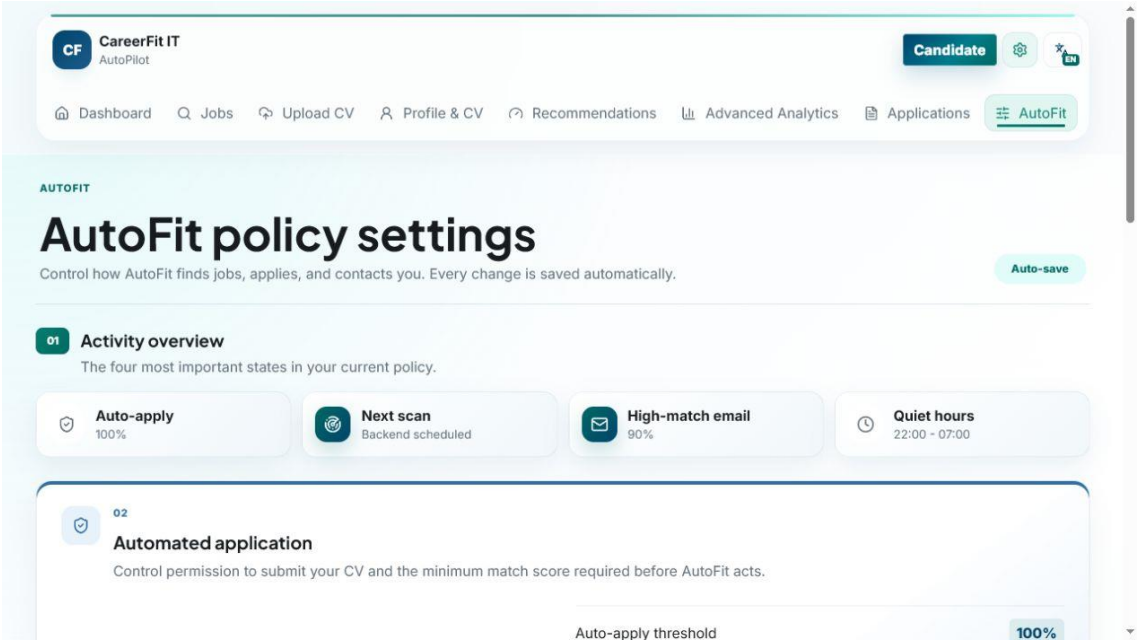


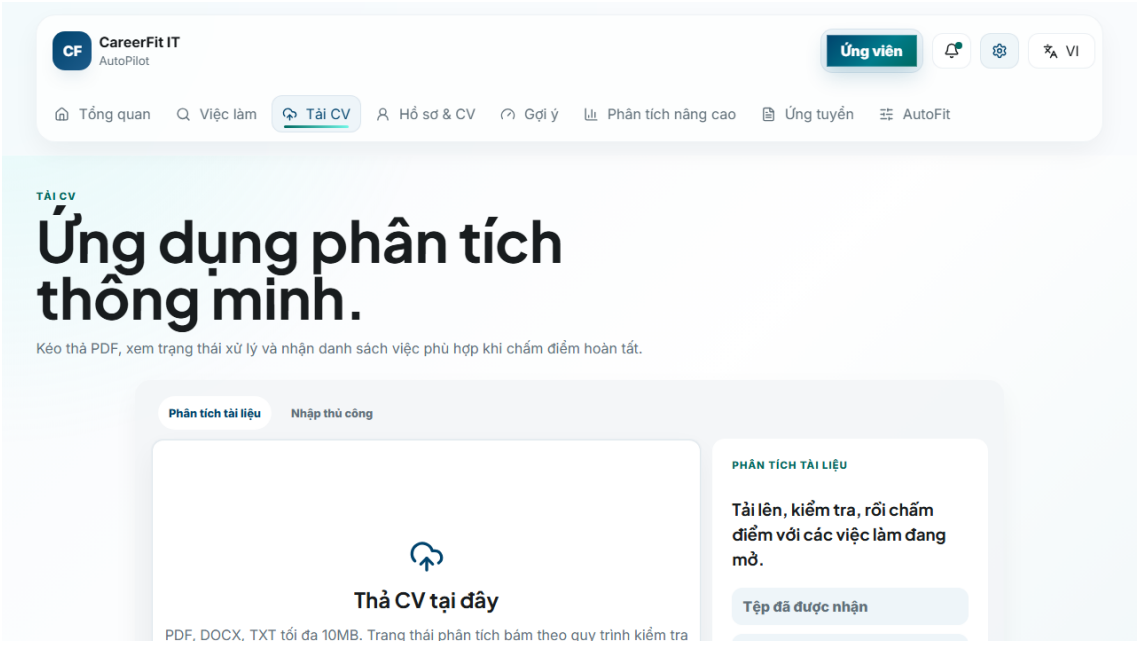
Figure 3.10. Frontend routes and API data flow



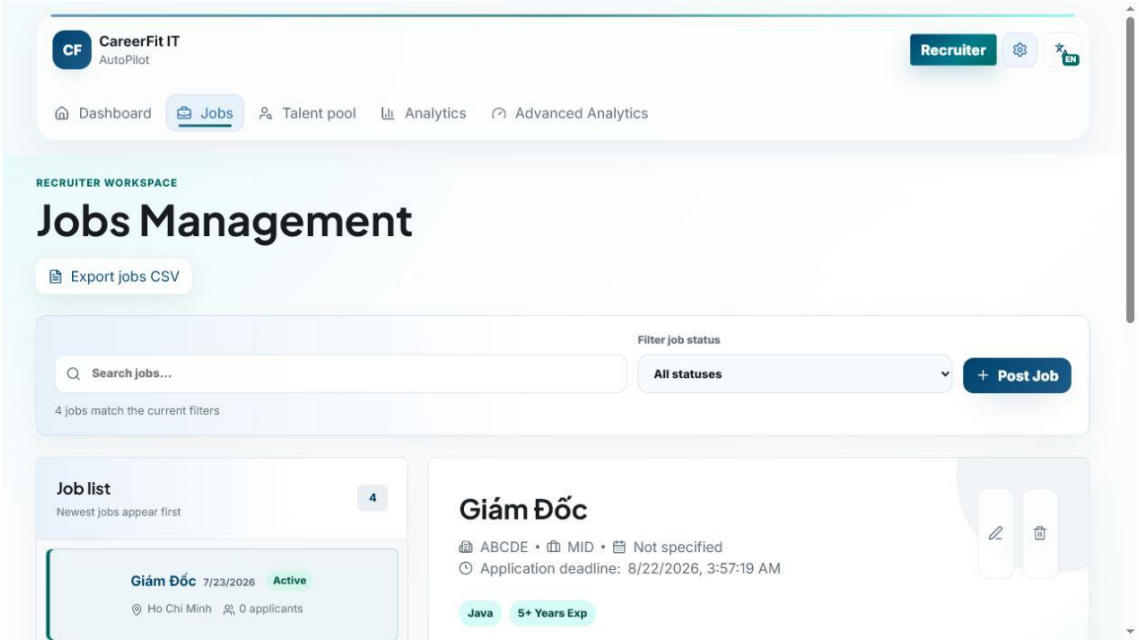
Screen 3.1. Candidate urgent-job catalogue



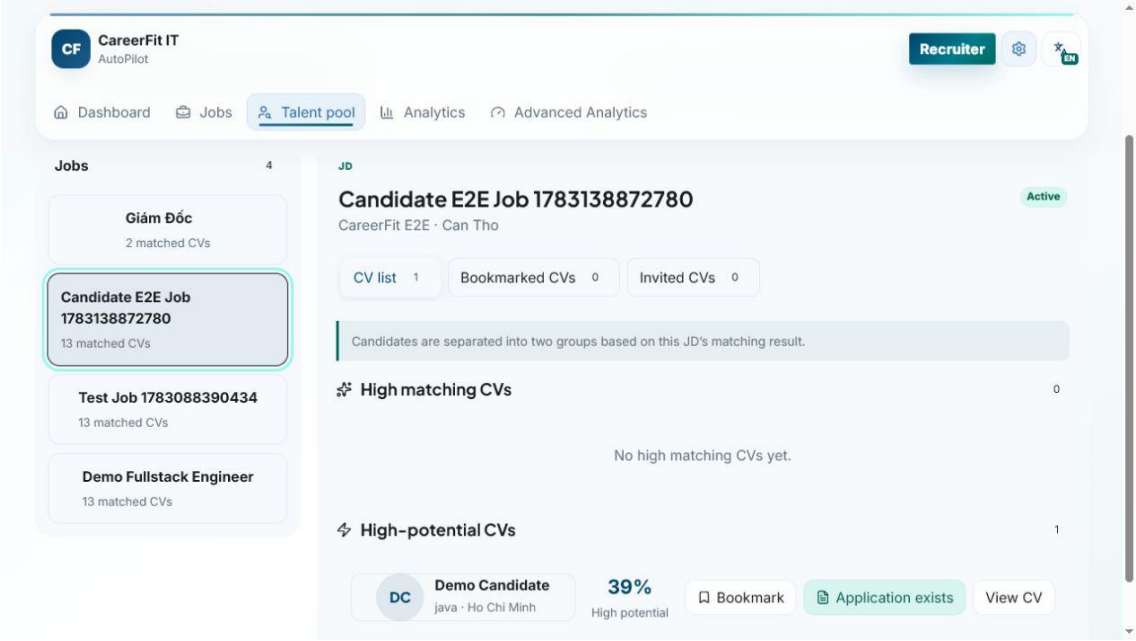
Screen 3.2. Candidate AutoFit policy settings



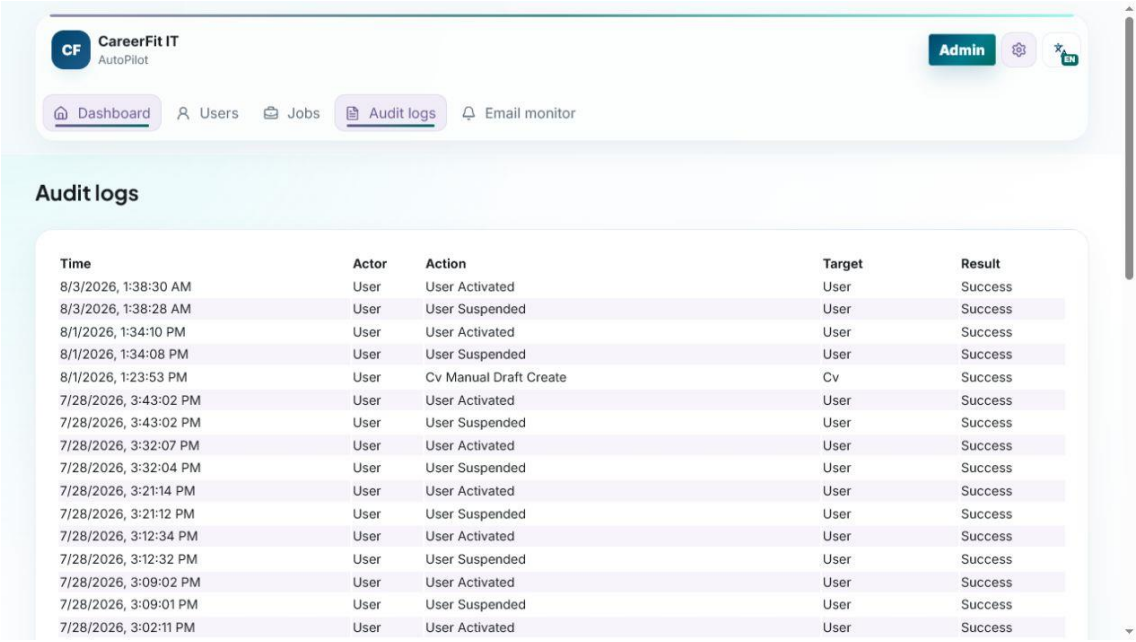
Screen 3.3. Candidate CV upload entry point



Screen 3.4. Recruiter Jobs and applicant workspace



Screen 3.5. Recruiter Talent Pool and Potential CV



Screen 3.6. Administrator audit logs

3.12 Deployment and Observability Implementation

The default local runtime starts PostgreSQL through Docker Compose and can start the backend under the backend profile. Host execution connects to localhost:5433; container execution connects to postgres:5432. The backend image mounts /app/storage/cv, and environment variables override database, JWT, CORS, application URL, mail, OCR, and storage configuration. The frontend development server runs on 127.0.0.1:5173 and uses /api or a configured API base URL.

Actuator exposes health and Prometheus endpoints. Micrometer records HTTP request metrics with the application name tag and latency histograms. Console logs include the timestamp, thread, level, request ID when available, logger, and message. These features provide basic monitoring data, but an available endpoint does not by itself prove that the monitoring system works. Chapter 4 must separately check reachability, access rules, Prometheus collection, and dashboard or alert evidence.

Maven builds the backend and can execute JUnit, Spring Security, and Testcontainers tests. The frontend build runs TypeScript checking before Vite bundling. Playwright configurations support local and production-oriented E2E runs. Build and test commands, versions, environment, failures, and generated artifacts must be recorded when Chapter 4 results are finalized.

3.13 Implementation Limitations

Table 3.6. Verified implementation limitations

Area	**Current limitation**	**Consequence or required improvement**
Email actions	Hashed token and confirm-then-POST flow	Add rate limiting, delivery monitoring, and deployment-specific origin controls
CV/OCR	Preprocessing and cleanup are heuristic	Scanned layouts and low-quality images still need user correction
Text processing	Simple tokenization remains the direct-score baseline	Technology spelling and Vietnamese phrases may still lose information
Potential model	Versioned aliases and transfer weights are manually defined	Validate rules with recruiter-labeled data and review model drift
Labels	Direct labels use only the medium/high boundaries	Align configuration names and UI wording with tested boundaries
Automation	Many policy guards and schedules interact	Add more time-zone, quota, cooldown, deadline, and concurrency tests
Frontend session	JWT and account summary use sessionStorage	Evaluate stronger refresh/cookie architecture and XSS controls
File storage	Local path or Docker volume	Add malware scanning, encryption, backup, retention, and protected object storage
Audit	Several services write audit rows directly	Centralize redaction, request context, schema rules, and integrity policy

These limitations are included because the purpose of an implementation chapter is to explain the system that exists, not an idealized version. They also provide concrete hypotheses and negative cases for Chapter 4.

3.14 Chapter Summary

This chapter explained the current implementation in Spring Boot, React, PostgreSQL, and Flyway. It covered password-based JWT security, CV extraction and human review, direct matching, skill-transfer Potential assessment, feedback learning, recruiter company and Talent Pool workflows, applications, account-level AutoFit controls, email actions, frontend integration, and monitoring. Chapter 4 evaluates the refreshed system.

CHAPTER 4. TESTING AND EVALUATION

4.1 Evaluation Objectives

The evaluation was designed to answer four questions. First, does lexical scoring with Rocchio produce repeatable results and move holdout items in the expected direction after feedback? Second, do the backend modules and database migrations pass the unit, integration, security, and contract tests? Third, can the main Guest, Candidate, Recruiter, and Administrator workflows run through the integrated browser application? Fourth, what can be measured about local API latency, authorization, health, and monitoring without presenting the results as production validation?

The evaluation treats algorithm results, software correctness, browser workflows, security checks, and system health as separate areas. Passing one area does not mean that every other area also passes. In particular, the controlled Rocchio benchmark does not measure hiring quality with real production recruitment data, and a passing backend test suite does not prove that browser, monitoring, or deployment work is complete.

Backend, algorithm, frontend build, and integrated Chrome results were refreshed on August 3, 2026 (ICT). The observed Git HEAD was 242e13a8f7d16fc9ebcab9780264c2c2b2b4ef06, but the worktree contained 197 modified or untracked entries. The results therefore describe this working tree and cannot be reproduced from the commit alone unless the full source state is preserved.

4.2 Evaluation Environment

Table 4.1. Evaluation environment

Component	Observed version or configuration
Host operating environment	Windows, PowerShell
Java	21.0.1
Backend framework	Spring Boot 3.2.5
Maven	Repository Maven Wrapper; clean verify
Node.js / npm	Bundled Node runtime with repository npm scripts
Frontend build	Vite 6.4.3, React 18.3, TypeScript 5.9, ESLint
Docker / Compose	Docker Desktop with Compose
Test database	PostgreSQL 16 through Testcontainers; local Compose PostgreSQL for E2E
Database migrations	Flyway V1–V24
Browser E2E	Playwright 1.61 using installed desktop Chrome
Integrated backend	Compose backend on port 8080
Integrated frontend	Vite development server on 127.0.0.1:5173

Docker Desktop and Compose provided PostgreSQL and the backend for integrated browser verification. Testcontainers created isolated PostgreSQL 16 databases for backend integration tests. The browser suite used the local CareerFit database, which contained imported, seeded, and E2E records. The public interface showed 992 open Jobs during the screenshot session. This runtime data is separate from the controlled algorithm dataset.

Earlier commands are recorded in evidence/CHAPTER5_EVIDENCE_20260703.md. For the August 3 refresh, Surefire XML, Maven output, TypeScript/ESLint/Vite results, Playwright output, the current screenshots, and evaluation/result.json are the main evidence. The report states the working-tree limitation instead of treating the commit hash as a complete release identifier.

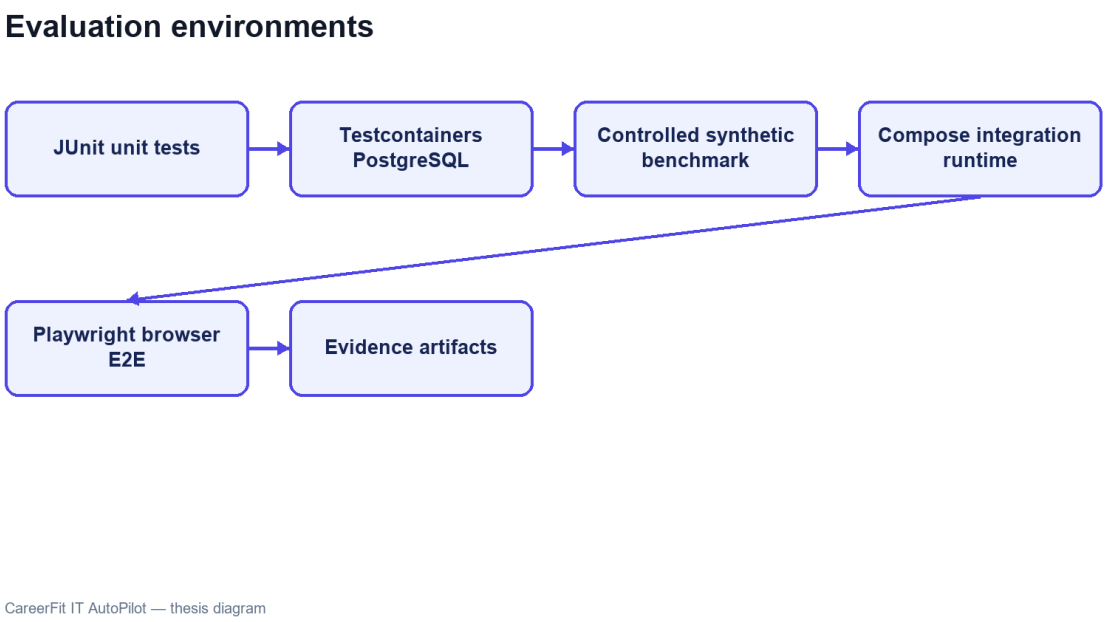


Figure 4.1. Evaluation environments and evidence sources

4.3 Dataset and Evaluation Design

4.3.1 Controlled Algorithm Dataset

AlgorithmEvaluatorTest loads evaluation/controlled-dataset.json. The dataset contains 50 Jobs and 100 unique CVs across IT domains. It defines 300 training pairs and 300 holdout pairs. The file hash observed during every repeated run was 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcf89fcc5cc2f2.

The benchmark is intentionally synthetic. Each Job is associated with a training-positive CV and a separate holdout-positive CV sharing a latent skill that is not emphasized in the original JD. The baseline ranks candidates using the original

lexical representation. A positive feedback event supplies evidence from the training CV; Rocchio updates the Job vector; and the system reranks candidates. Metrics are then computed on holdout relevance rather than using the same CV that generated feedback as the only success case.

This design checks whether a defined feedback signal moves a related holdout item upward. It does not reproduce real recruiter behavior, naturally unbalanced applicant pools, demographic differences, intentionally misleading CVs, or changes in labor-market terms. The dataset includes a designed shared feature that makes a large improvement possible, so the result should not be treated as the expected effect in production.

4.3.2 Local Runtime Data

The integrated runtime used Flyway seed data and previously imported Job records. It supports realistic API volume and complete application workflows, but it is not a labeled relevance dataset. Consequently, it is suitable for functional/E2E and small latency checks, not for calculating ranking precision or fairness. E2E uses demo Candidate, Recruiter, and Administrator accounts and changes local database state.

4.4 Backend Automated Testing

The complete backend suite was executed with .\mvnw.cmd clean verify. It compiled 142 application source files and 35 test source files, started PostgreSQL 16 Testcontainers, applied Flyway migrations V1–V24, executed 33 Surefire suites, and built the backend JAR.

Table 4.2. Backend automated test results

Measure	**Result**
Test suites	33
Tests	131
Failures	0
Errors	0
Skipped	0
Aggregated Surefire suite time	102.194 s
Maven lifecycle	clean verify and JAR build passed

The suites covered application context, API contracts, security, account login, CV drafts and review, OCR/extraction, skill suggestions and transfer rules, Job quality and lifecycle, server-side catalogues, matching, feedback, automation, deadlines, bookmarks, invitations, analytics, settings, and the controlled AlgorithmEvaluatorTest.

All 131 registered JUnit tests passed on August 3, with no failures, errors, or skips. The aggregated Surefire suite time was 102.194 seconds. Negative tests still

produced expected handled errors in their own cases, but the Maven build completed successfully and no final test report contained a failure.

4.5 Controlled Algorithm Results

The benchmark used Rocchio parameters α=1.0, β=0.75, and γ=0.15. Coverage remained 1.0 before and after feedback. Table 4.3 reports the fresh artifact values.

Table 4.3. Baseline and Rocchio results on the synthetic holdout scenario

Metric	Baseline	After Rocchio	Delta
Precision@5	0.012000	0.172000	+0.160000
Recall@5	0.060000	0.860000	+0.800000
nDCG@3	0.030000	0.830000	+0.800000
nDCG@5	0.037737	0.837737	+0.800000
nDCG@10	0.050424	0.850424	+0.800000
MRR	0.058755	0.842665	+0.783910
HitRate@5	0.060000	0.860000	+0.800000
Coverage	1.000000	1.000000	0

The increase across position-sensitive metrics shows that the controlled positive feedback moved relevant holdout CVs toward the top of the ranking. Precision@5 reaches 0.172 after feedback, so the top-five lists are not composed entirely of labeled relevant items. Recall@5 and HitRate@5 reach 0.86 in the designed scenario, while 14 percent of query cases still do not place the expected holdout item within the first five results.

The August 3 full-suite run reproduced dataset hash 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcf89fcc5cc2f2. It produced baseline nDCG@5 of 0.037737056145, Rocchio nDCG@5 of 0.837737056145, and a delta of 0.80. These values describe the controlled dataset and should not be read as expected improvement on real recruitment data.

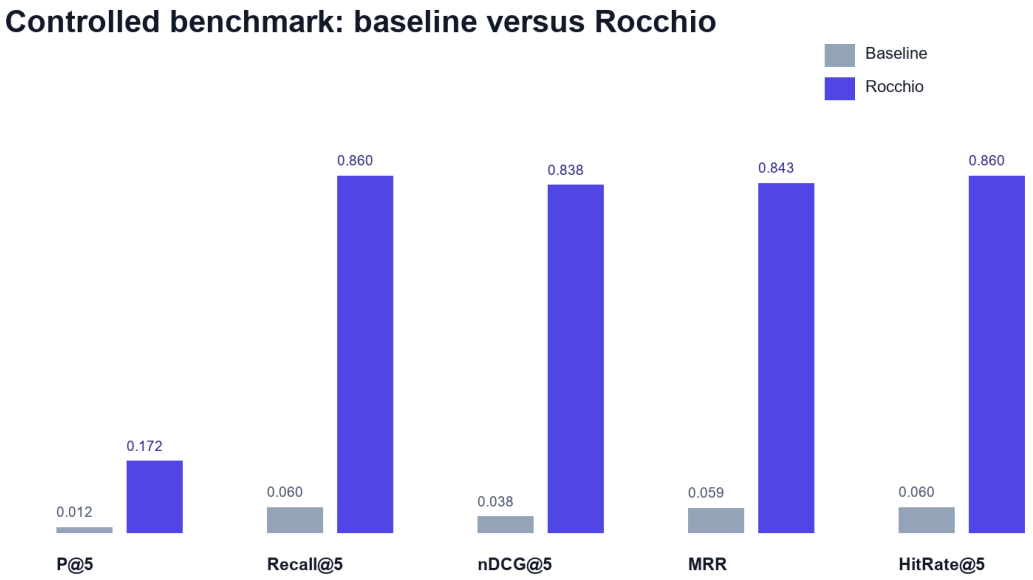


Figure 4.2. Baseline and Rocchio benchmark metrics at K = 5

4.6 Concurrency Observation in Benchmark Runs

The final isolated benchmark completed without StaleObjectStateException. Feedback learning is registered after transaction commit, and benchmark setup removes prior Matchings in bulk and clears the persistence context before recomputation. This makes the repeated baseline-versus-learned measurement deterministic within the controlled test environment.

Earlier benchmark runs showed a background database conflict even though the main test assertions passed. The current run did not log that exception because feedback learning and matching now start after the first transaction commits, and the benchmark clears previous Matching records before recalculation. This remains important: both test assertions and background errors must be checked.

The implemented correction controls asynchronous transaction timing and checks the final logs for background failures. Production work should still define retry, conflict, timeout, and recovery policies, and CI should continue to fail when uncaught background exceptions appear even if foreground JUnit assertions pass.

4.7 Frontend Build Evaluation

The frontend verification ran TypeScript checking with no output, ESLint, Vite 6.4.3 production build, and the bundle check. The August 3 build completed successfully and transformed 2,361 modules.

Table 4.4. Frontend build artifacts

Artifact	Uncompressed size	Gzip size

Artifact	Uncompressed size	Gzip size
index.html	1.37 kB	0.67 kB
Main CSS	132.25 kB	24.36 kB
Icons chunk	22.10 kB	5.03 kB
Query chunk	39.78 kB	12.20 kB
React chunk	181.35 kB	59.62 kB
Application chunk	310.48 kB	83.45 kB
Charts chunk	387.39 kB	112.17 kB

Manual Rollup chunking separated React, query, chart, and icon dependencies. The largest generated JavaScript chunk was charts at 387.39 kB (112.17 kB gzip), below Vite's 500 kB warning threshold. This is build evidence only; no browser performance profile was collected.

4.8 Browser End-to-End Evaluation

The Playwright suite ran with the Vite frontend, the current backend, PostgreSQL, and the installed desktop Chrome channel. All 41 tests across workflow, backend-contract, account-state, catalogue, and resilience coverage passed in 42.1 seconds.

Table 4.5. Integrated Chrome workflow results

Scenario group	Main verification	Result
Guest and authentication	Public catalogue/detail, email/password login, session restoration, and role routes	Passed
Candidate CV and matching	Manual draft, upload/review/confirm, polling, match cards, filters, and feedback	Passed
Candidate recruitment	Urgent catalogue, apply/withdraw/history, settings, and AutoFit controls	Passed
Recruiter Jobs	Company onboarding, quality preview, draft/publish, deadline, applicants, and catalogue	Passed
Recruiter Talent Pool	Potential view, Candidate details, bookmark, invitation, and state changes	Passed
Administration and resilience	Suspend/reactivate, audit, API errors, navigation, account state, and dark mode	Passed

The result covers Guest search and details; email/password login; Candidate CV draft/review, matching, urgent filters, applications, settings, and AutoFit; Recruiter company onboarding, Job draft/publish, applicants, Talent Pool, bookmarks, and invitations; Administrator operations; session restoration; server-side catalogues;

retryable errors; dark mode; and role navigation. It is not an independent UAT study, and Firefox/WebKit were not run in this refresh.

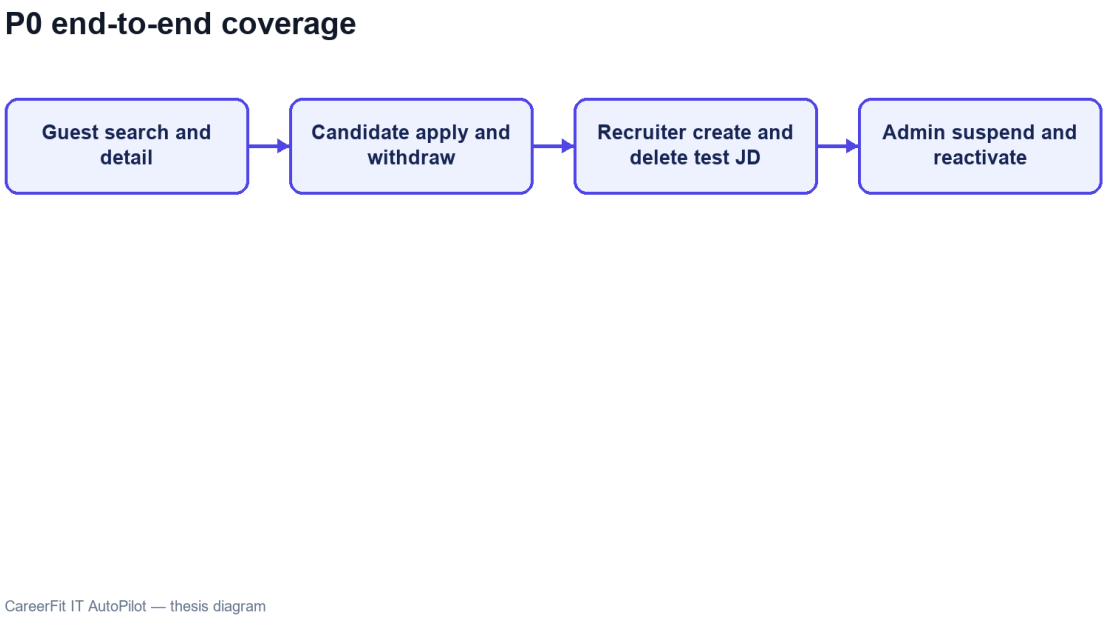


Figure 4.3. P0 end-to-end workflow coverage

4.9 Security-Oriented API Checks

Small negative checks were executed against the integrated backend to verify public access, missing authentication, invalid authorization scheme, valid Candidate authentication, and role denial.

Table 4.6. API authorization observations

Request	Expected	Observed	Result
Public Job search without token	200	200	Passed
Candidate CV endpoint without token	401	401	Passed
Current-account endpoint with invalid Basic scheme	401	401	Passed
Current-account endpoint with valid Candidate bearer token	200	200	Passed
Administrator dashboard with Candidate bearer token	403	403	Passed

These checks confirm selected URL-layer outcomes, not a complete penetration test. They do not evaluate token theft, cross-site scripting, CV malware, rate limiting, secret rotation, SQL-injection tooling, or every ownership boundary. The

email-action flow now uses hashed storage and confirm-then-POST redemption, but the broader production-security topics remain outside the verified evidence.

4.10 Runtime Health, Monitoring, and Local Latency

The public Job API returned HTTP 200. The aggregate /actuator/health endpoint and the liveness and readiness groups returned HTTP 200 with status UP. Mail health is disabled when application mail is disabled, preventing an intentionally absent local mail provider from making aggregate health misleading. Prometheus metrics remained available in the local evaluation profile.

Table 4.7. Runtime observations

Endpoint or check	Observation
/api/jobs/search?page=0&size=20	200
/actuator/health	200, UP
/actuator/health/liveness	200, UP
/actuator/health/readiness	200, UP
/actuator/prometheus	200

The final runtime check returned HTTP 200 and status UP for aggregate health, liveness, and readiness. Mail health is disabled when application mail is disabled, so the absence of a local mail provider no longer makes aggregate health misleading. Production monitoring would still require protected component details, alerting, and an external mail check when email is enabled.

Thirty sequential warm requests to the public Job search endpoint produced a mean of 61.79 ms, p50 of 55.20 ms, p95 of 85.11 ms, minimum of 44.99 ms, and maximum of 99.32 ms. This was a single-client localhost sample without concurrent load, controlled warm-up, network latency, repeated batches, or resource monitoring. It demonstrates only that the observed local endpoint responded consistently in this small sample; it is not a throughput or capacity benchmark.

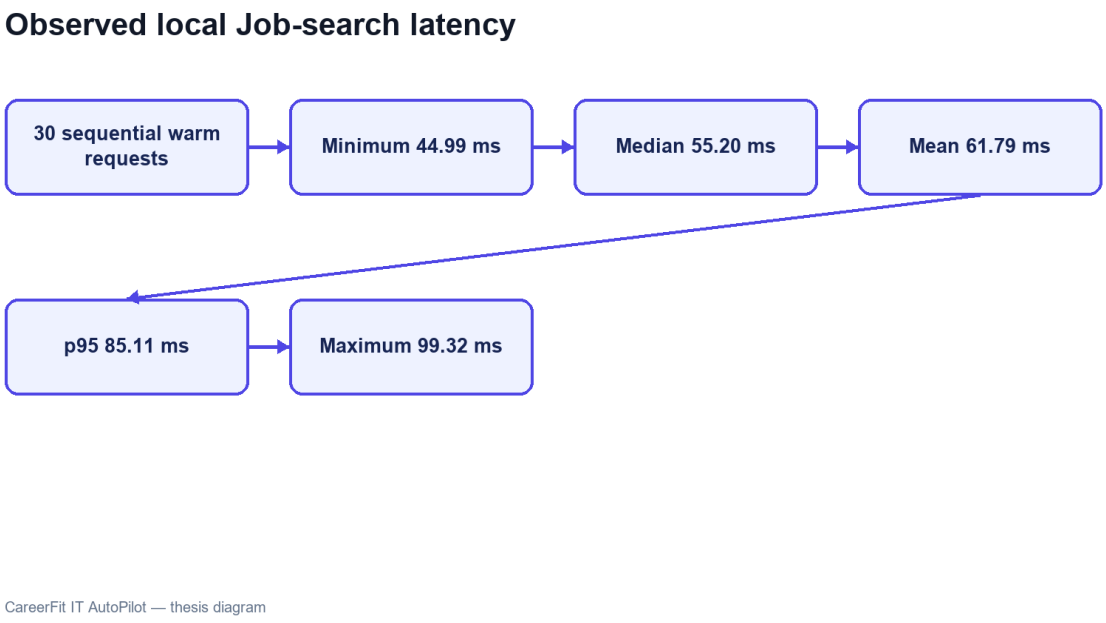


Figure 4.4. Local Job-search latency distribution

4.11 Evaluation Summary by Research Question

Table 4.8. Answers supported by current evidence

Question	Evidence-supported answer
Does Rocchio change holdout ranking in the intended direction?	Yes for the controlled synthetic dataset; the same large metric delta was reproduced.
Is the backend automated suite passing?	Yes. All 131 tests across 33 suites passed, and clean verify built the JAR.
Do the main browser workflows execute?	Yes. All 41 integrated Chrome tests passed; Firefox/WebKit and participant UAT remain incomplete.
Are selected authorization boundaries enforced?	Observed public, authentication, and role checks returned expected statuses; this is not comprehensive security validation.
Is the runtime healthy?	Yes in the evaluated local profile: aggregate health, liveness, readiness, and the core API returned HTTP 200.
Is performance production-ready?	Not evaluated. Only a small sequential localhost latency sample was collected.

4.12 Threats to Validity

4.12.1 Construct Validity

The synthetic benchmark measures retrieval behavior for planted relevance relationships, not hiring success, candidate quality, fairness, or recruiter satisfaction. Matching metrics cannot establish whether a person should be employed. Scripted E2E success measures workflow execution, not usability or user trust.

4.12.2 Internal Validity

The benchmark runs inside a Spring context, so asynchronous work and persisted Matching records can affect repeatability if they are not controlled. The final test registers learning after commit, clears previous Matching state before recomputation, and checks logs for background failures. Browser tests use explicit API and UI conditions, avoid relying only on the first seeded Job, and delete the Job created by the Recruiter flow.

4.12.3 External Validity

The controlled data lacks organic recruiter judgments, changing terminology, demographic diversity, adversarial behavior, and production class imbalance. The runtime is one Windows workstation with Docker Desktop, one local PostgreSQL instance, and desktop Chrome. Results cannot be generalized to enterprise load, other browsers, cloud deployment, or a population of real users.

4.12.4 Reproducibility

The dataset hash and generated JSON help reproduce the algorithm experiment. Maven Wrapper, Flyway, Testcontainers, the package lock, and recorded commands help rebuild the environment. Recruiter E2E cleanup is automatic, but exact reproduction is still harder because the working tree is not clean, the local database can change, imported records remain, and some web assets are external. The final thesis release should use a clean commit or archive together with a fixed evidence package.

4.13 Chapter Summary

The August 3 evaluation passed all 131 backend tests, completed TypeScript, ESLint, production-build, and bundle checks, and passed all 41 integrated Chrome tests. The controlled Rocchio benchmark kept its expected synthetic improvement. These results support a functioning academic prototype, but not production readiness or proven hiring effectiveness.

CONCLUSION

1. Summary of Results

CareerFit IT AutoPilot was implemented as an IT recruitment prototype with public Job discovery, password-based role workspaces, reviewed CV processing, direct matching, a separate Potential assessment, recommendations, Rocchio feedback, applications, Recruiter Talent Pool features, account-level AutoFit policies, email actions, analytics, and administrative monitoring.

The refreshed evaluation provides evidence for several technical outcomes. The backend passed 131 tests across 33 suites. TypeScript checking, ESLint, Vite production compilation, and the bundle check passed. All 41 integrated Chrome tests passed across role workflows, backend contracts, account state, catalogues, and resilience, including the new CV review, Job draft/publish, urgent, Talent Pool, bookmark, invitation, and policy-setting flows.

The controlled Rocchio benchmark showed the expected behavior on its synthetic dataset. With 50 Jobs, 100 unique CVs, 300 training pairs, and 300 holdout pairs, nDCG@5 increased from 0.037737 to 0.837737, Recall@5 and HitRate@5 increased from 0.06 to 0.86, and MRR increased from 0.058755 to 0.842665. These results show adaptation in the designed synthetic scenario, but they do not estimate performance on real recruitment data.

The August 3 refresh verified the larger project state, kept the largest JavaScript chunk below 500 kB, removed stale passwordless claims, updated the CV review and skill-transfer design, expanded browser coverage to 41 tests, and replaced six report interface images with current screens. The result is suitable for a local thesis demonstration, not a production release.

Table C.1. Consolidated result status

Evaluation area	Supported result	Qualification
Backend automated tests	131/131 tests across 33 suites passed	clean verify and JAR build completed
Controlled feedback benchmark	Large, deterministic Rocchio improvement	Synthetic causal design; not production effectiveness
Frontend checks	TypeScript, ESLint, Vite build, and bundle check passed	Largest JS chunk 387.39 kB; no Vite size warning
Browser workflows	41/41 integrated Chrome tests passed	No Firefox/WebKit or independent participant UAT
Authorization spot checks	Observed 200/401/403 outcomes matched expectations	Not a complete security assessment
Runtime APIs	Core Job API and health endpoints returned HTTP 200/UP	Local profile only; no production monitoring

Evaluation area	Supported result	Qualification
Local latency sample	Mean 61.79 ms and p95 85.11 ms for 30 sequential Job queries	No concurrency, controlled load, or production network

2. Achievement of Thesis Objectives

2.1 Role-Based Recruitment Platform

The role-based workflow objective was achieved at prototype level. Guests can browse public Jobs and employers. Candidates can review CVs, inspect matching and recommendation results, apply or withdraw, submit feedback, and configure AutoFit. Recruiters can complete a company profile, manage Jobs, review applicants and the Talent Pool, bookmark or invite candidates, and update application states. Administrators can monitor users, Jobs, email actions, and audit records. The 41 Chrome tests cover representative flows but not every route or browser.

2.2 CV and Job-Description Processing

The project implements uploaded and manually created CVs, explicit processing statuses, file-type and magic-byte validation, PDF and DOCX extraction, image and scanned-PDF OCR fallback, sparse-text warning, failure reason persistence, and structured quality signals. Job creation includes structured fields and conditional validation. This objective is supported by source inspection and automated tests for extraction, validation, Job service behavior, and integration contracts.

The implementation is limited by local storage, the availability of external Tesseract software, page and timeout limits, and a simple deterministic tokenizer. It should be described as a working ingestion pipeline rather than a general document-understanding system.

2.3 Matching and Recommendation

CareerFit implements a static-corpus TF-IDF vectorizer, cosine similarity, normalized score, LOW/MEDIUM/HIGH labels, potential heuristics, and shared-term reasons. CV–JD matching and profile-oriented recommendation remain separate workflows, and Matching state remains separate from Application state. Scoring, TF-IDF, batch isolation, and API contract tests passed.

The objective was achieved as an interpretable lexical baseline, not as a semantic matching model. Technology names with punctuation, synonyms, Vietnamese phrases, career changes, and context beyond shared terms remain limitations. The displayed percentage is a normalized similarity score, not a tested probability of recruitment success.

2.4 Feedback Learning

The system records GOOD_MATCH, POTENTIAL, BAD_MATCH, and NOT_INTERESTED feedback with actor and channel information. Positive and negative learning signals update the Job representation using Rocchio with $\alpha=1.0$, $\beta=0.75$, and $\gamma=0.15$. Recalculation begins from the original Job vector and current feedback history, which produces repeatable results for the same inputs. Affected Matchings are marked stale and rescored asynchronously.

The controlled benchmark confirms the intended Rocchio behavior in the planted latent-skill scenario. The final run completed without the earlier optimistic-lock exception after learning was moved to an after-commit boundary and benchmark state was cleared before recomputation. nDCG@5 and HitRate@5 each increased by 0.80, although this does not prove reliability under production concurrency.

2.5 Policy-Driven Automation and Email Action

AutoFit policies store thresholds, enablement, notification preferences, cooldown, quota, quiet-hour, timezone, digest, and pause settings. Scheduled services recompute stale results, send digests and high-match notifications, clean expired email actions, and execute auto-apply. Auto-apply validates the default CV, Job status, threshold, and duplicate state and limits creation to three applications per run.

This objective is achieved as configurable prototype automation. Notification policy and application authorization remain separate control paths, scheduler timing is externalized through app.scheduler.*, and the email-action channel uses hashed token storage with GET confirmation followed by POST execution. Further production work is still required for rate limiting, secret rotation, mail delivery, and deployment-specific origin controls.

2.6 Auditability and Evaluation

Major application, feedback, automation, and administrative actions create structured audit rows. Notification delivery and email-action statuses add operational evidence. The project includes unit, integration, security, algorithm, and E2E tests and produces a dataset-hashed benchmark artifact.

Audit coverage has not been formally proven for every state-changing endpoint, and direct repository calls can still produce inconsistent metadata conventions. The evaluation nevertheless checked logs and side effects in addition to test exit codes, which helped detect and correct the earlier concurrency and health problems.

Table C.2. Objective assessment

Objective	Assessment	Primary evidence

Objective	Assessment	Primary evidence
Role-based web platform	Achieved for prototype scope	API contracts and 41 integrated Chrome tests
Reviewed CV/JD ingestion	Achieved for supported formats and configured OCR	Draft/review/confirm, extraction, quality, Job, and integration tests
Explainable matching and Potential	Achieved as rule-based baseline	TF-IDF/scoring tests and versioned skill-transfer reasons
Rocchio feedback adaptation	Achieved in controlled test; production concurrency unverified	Synthetic benchmark, after-commit execution, and clean reports
AutoFit and communication	Achieved as configurable prototype	Account policies, scheduler, reminders, notifications, and signed email actions
Recruiter operational workflow	Achieved for prototype scope	Company, Job draft/publish, applicants, Talent Pool, bookmark, and invitation tests
Production readiness	Not demonstrated	Local health passed; scale, real users, and production security remain missing

3. Discussion

3.1 Value of an Interpretable Baseline

CareerFit uses a lexical vector-space baseline instead of assuming that a dense or generative model is always better. This choice lets users inspect the input tokens, weights, shared terms, feedback centroids, and score calculation. For an academic system, this transparency helps explanation, debugging, boundary testing, and presentation of the implementation.

The cost is reduced semantic capability. A model that cannot reliably equate related technologies or interpret career context may under-rank suitable candidates and over-rank documents containing repeated keywords. The correct conclusion is not that TF-IDF is sufficient for all recruitment, but that it provides a reproducible baseline against which future semantic or hybrid retrieval can be compared.

3.2 Human-in-the-Loop as System Structure

Human-in-the-Loop in CareerFit is expressed through multiple control points: users supply and validate data, inspect reasons, configure policy, apply or invite, provide feedback, pause automation, and review audit history. The system distinguishes score evidence from Application state and policy action. This gives users more control than adding a general approval button after an automated decision that they cannot understand.

However, human oversight does not automatically make a system fair or safe. Users may accept misleading explanations, policies may use unsuitable thresholds, and feedback may repeat individual or organizational bias. HITL must therefore be

combined with error analysis, access control, ways to question or appeal results, audit review, and evaluation with representative data.

3.3 Meaning of the Rocchio Improvement

The large benchmark delta is expected from the experimental design: a latent skill is deliberately shared between a training-positive CV and a holdout-positive CV but omitted from the initial JD emphasis. Rocchio moves the Job vector toward the training example, making the holdout example easier to retrieve. Repeated equality of the metrics shows implementation determinism under this scenario.

The result does not show that every real Candidate feedback event improves ranking by 0.80 nDCG@5. Organic feedback may be inconsistent, sparse, biased, delayed, or based on salary and location rather than technical relevance. Production evaluation would require independently labeled data, time-based splits, multiple reviewers, disagreement analysis, and online or offline comparison against a baseline without feedback.

3.4 Test Results versus Background Errors

The earlier background StaleObjectStateException showed why test counts cannot be the only release check. JUnit assertions can pass while asynchronous tasks log failures outside the main test flow. Component health and overall health must also be read together. A reliable system report should check logs, health components, output files, and side effects in addition to exit codes.

Future CI should continue to fail on uncaught background exceptions, control schedulers during algorithm tests, wait for asynchronous work explicitly, and archive logs as first-class results. Operational health details should be protected while remaining available to authorized operators.

3.5 Product Positioning

CareerFit should be positioned as an IT-focused, explainable, policy-controlled academic recruitment prototype. It demonstrates how Job portal functions, ranking, recommendation, feedback, automation, email actions, and audit can be connected. It should not be positioned as having better general-purpose AI than commercial recruiter platforms or as replacing professional recruitment judgment.

The clearest difference is workflow control: matching and recommendation are separate, policies are explicit, users keep control points, feedback has a visible mathematical effect, and actions can be audited. This difference is limited in scope but is well supported by the implementation.

4. Limitations

4.1 Data and Algorithm Limitations

The controlled dataset is synthetic and designed to show how feedback changes ranking. It does not include normal language variation, recruiter disagreement, demographic analysis, misleading CVs, or changing market terms. TF-IDF still uses a static IT corpus and simple tokenization. The Potential assessment now uses a versioned skill-transfer knowledge base, but its aliases, transfer weights, family rules, and guard thresholds are still manually defined.

4.2 Evaluation Limitations

The 131 backend tests do not prove complete path coverage. Browser evaluation covers 41 automated cases in desktop Chrome only, and no independent users participated. Security checks were targeted rather than a penetration test, while the latency sample used 30 sequential local requests. Concurrency, capacity, cross-browser behavior, accessibility with users, and real hiring outcomes remain unevaluated.

4.3 Security and Privacy Limitations

Notification action tokens are hashed, and state changes require POST after confirmation. Access tokens are stored in frontend sessionStorage rather than persistent localStorage. Local CV storage lacks documented malware scanning, encryption, retention enforcement, and recovery testing. Public Swagger and Prometheus access are suitable for local demonstration but should be restricted in production. Privacy, fairness, and management of users' personal data have not been validated with a real deployment population.

4.4 Reliability and Maintainability Limitations

The current implementation improves earlier gaps through after-commit matching, CV review before scoring, versioned Potential rules, server-side catalogues, recruiter company onboarding, Job drafts, bookmarks, policy guards, and broader E2E coverage. Audit records are still written directly by several services, and the evaluated worktree is not a fixed release commit. These limits make exact reproduction and production operation harder.

Table C.3. Principal limitations and impact

Limitation	Impact on conclusions
Synthetic benchmark	Supports causal behavior only, not real hiring effectiveness
Lexical direct score	Limits semantic and contextual understanding
Manual skill-transfer knowledge base	Potential rules require labeled validation and ongoing maintenance

Limitation	Impact on conclusions
Production concurrency not evaluated	Conflict, retry, and recovery behavior remain unproven at scale
Local-only health verification	Does not establish deployed availability
Chrome-only 41-test automation suite	Limits cross-browser and real-user usability conclusions
Email delivery and abuse controls not production-tested	Signed action flow exists, but real delivery and rate limits remain unverified
Dirty worktree and mutable E2E database	Weakens exact experiment reproduction

5. Future Work

Future security work should add rate limiting, stronger session handling, protected management endpoints, malware scanning, encrypted CV storage, retention rules, and tested backup recovery. Email delivery should be tested with a real provider, including expiry, replay, scanner behavior, and failure recovery. CI should continue checking background logs and asynchronous completion rather than relying only on foreground test status.

The matching baseline can then be extended using a hybrid approach. Domain-aware tokenization and aliases should first address punctuation-sensitive technologies and Vietnamese phrases. Sparse lexical scores can be combined with sentence or skill embeddings and structured constraints. Any new model should be evaluated against the current baseline on a frozen, independently labeled dataset rather than assumed superior because it is more complex.

Evaluation should expand to recruiter-labeled CV–JD pairs, temporal holdout sets, inter-rater agreement, hard-negative analysis, subgroup/fairness checks, and calibration of labels. A controlled user study should measure task completion, time, comprehension of explanations, trust calibration, and ability to override automation. Cross-browser E2E, load testing, failure injection, backup/restore, email delivery, and security testing should be included in release evidence.

Operational development should move CVs to protected object storage, define encryption and retention policy, centralize audit generation and redaction, strengthen token/session architecture, and associate each release with a clean commit and a fixed evidence archive. Integration with external ATS or Job boards should occur only after consent, ownership, error recovery, and audit contracts are defined.

6. Closing Remarks

CareerFit demonstrates controlled IT recruitment automation by connecting job discovery, explainable matching, profile-based recommendations, feedback learning, policy-driven actions, and audit records. It keeps scores, business state, and user decisions visible.

Local evaluation passed 131 backend tests, the frontend type/lint/build/bundle checks, 41 integrated Chrome tests, health checks, and the controlled Rocchio benchmark. This supports a tested Human-in-the-Loop prototype, not a system ready to make real hiring decisions.

REFERENCES

[1] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” Communications of the ACM, vol. 18, no. 11, pp. 613–620, 1975, doi: 10.1145/361219.361220.

[2] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.

[3] C. Buck and P. Koehn, “Quick and reliable document alignment via TF/IDF-weighted cosine distance,” in Proceedings of the First Conference on Machine Translation, vol. 2, Berlin, Germany, 2016, pp. 672–678, doi: 10.18653/v1/W16-2365.

[4] J. J. Rocchio, “Relevance feedback in information retrieval,” in The SMART Retrieval System: Experiments in Automatic Document Processing, G. Salton, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1971, pp. 313–323.

[5] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” ACM Transactions on Information Systems, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.

[6] N. Drushchak and M. Romanyshyn, “Introducing the Djinni Recruitment Dataset: A corpus of anonymized CVs and job postings,” in Proceedings of the Third Ukrainian Natural Language Processing Workshop, Torino, Italy, 2024, pp. 8–13, doi: 10.63317/2dcvy45ws6yb.

[7] X. Yu, R. Xu, C. Xue, J. Zhang, X. Ma, and Z. Yu, “ConFit v2: Improving resume-job matching using hypothetical resume embedding and runner-up hard-negative mining,” in Findings of the Association for Computational Linguistics: ACL 2025, Vienna, Austria, 2025, pp. 12775–12790, doi: 10.18653/v1/2025.findings-acl.661.

[8] S. Vaishampayan et al., “Human and LLM-based resume matching: An observational study,” in Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, NM, USA, 2025, pp. 4823–4838, doi: 10.18653/v1/2025.findings-naacl.270.

[9] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.

[10] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, Barcelona, Spain, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.

[11] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.

[13] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2006, doi: 10.6028/NIST.SP.800-92.

[14] A. Colas, J. Araki, Z. Zhou, B. Wang, and Z. Feng, “Knowledge-grounded natural language recommendation explanation,” in Proceedings of the 6th BlackboxNLP Workshop, Singapore, 2023, pp. 1–15, doi: 10.18653/v1/2023.blackboxnlp-1.1.

APPENDICES

Appendix A. Requirement-Test Traceability Matrix

The core traceability chain is: authentication and role controls to SecurityHardeningTest and P0 login flows; CV ingestion to service and integration tests plus the Candidate upload flow; matching and feedback to AlgorithmEvaluatorTest; Job lifecycle to the Recruiter create/verify/delete flow; administration to suspend/activate workflows; and operations to Actuator health, build, and runtime evidence.

Appendix B. Selected API Contracts

Representative endpoints are POST /api/auth/login; GET /api/jobs/search; POST /api/cv/upload; GET/PATCH /api/cv/{cvId}/review; POST /api/cv/{cvId}/review/confirm; GET /api/matches/me/cards; POST /api/matches/{matchingId}/feedback; POST /api/applications; GET /api/recruiter/talent/jobs/{jobId}/bookmarks; PATCH /api/automation/policy; GET then POST /api/email-action/redeem; and GET /actuator/health. Protected routes require a signed JWT and backend role/ownership checks.

Appendix C. Data Model Summary

Identity data is centered on UserAccount and role profiles. Recruitment data includes Job, CV with review fields, Matching, Application, RecruiterCvBookmark, Invitation state, Feedback, and the Skill catalogue. Automation and operations use AutomationPolicy, EmailAction, Notification, DeliveryLog, AuditLog, and scheduler state. Flyway V1–V24 records schema changes, and one-time email action tokens are stored as SHA-256 hashes.

Appendix D. Evaluation Summary

The final evidence package contains the 131-test backend reports, controlled algorithm benchmark, frontend type/lint/build results, 41-test Chrome output, runtime health evidence, updated screenshots, and evaluation/result.json. Chapter 4 reports what this evidence supports and what it does not support.

Appendix E. UAT and Demonstration Script

Preparation

1. Start PostgreSQL, the backend, and the frontend; verify aggregate health and the public Job API; and prepare separate Candidate, Recruiter, and Administrator accounts.
2. Record the current Git commit, working-tree status, configuration profile, database source, and evaluation artifact locations.
3. Use clearly marked test records and note the cleanup action for every Job, CV, application, invitation, or policy created during the demonstration.

Execution

1. As a Guest, search for an IT Job, apply filters, and open Job and employer details.
2. As a Candidate, sign in, upload a CV or save a manual draft, review and correct extracted sections, confirm the CV, wait for scoring, inspect match reasons and Potential explanations, filter urgent Jobs, apply or withdraw where permitted, and submit feedback.
3. As a Recruiter, complete the company profile, save a test Job draft, run the JD quality preview, publish it with an application deadline, inspect applicants and the Talent Pool, bookmark and invite a Candidate, update application status, and remove the test Job after verification.
4. Configure Candidate AutoFit thresholds, cooldown, quota, notification options, quiet hours, and human-control settings; then use run-now only for a controlled verification.
5. As an Administrator, inspect user, Job, audit, notification, and email-action records; suspend and reactivate a designated test account.
6. Show Actuator health, the archived backend and Chrome test results, the benchmark artifact, and the final cleanup state.

Acceptance and cleanup

1. Record each expected and observed result, preserve any failure evidence, and do not convert an unverified step into a passing claim.
2. Delete or deactivate demonstration data, restore account state, and confirm that background logs contain no unhandled exception.
3. Archive the final evidence with a clean commit or a complete source-state package.

Appendix F. Local Deployment Instructions

Prerequisites

- Install Java 21, Node.js with npm, Docker Desktop with Docker Compose, and Git.
- Provide environment-specific database, JWT, CORS, application URL, mail, OCR, and storage configuration. Do not commit production secrets.

Startup procedure

1. From the repository root, start PostgreSQL with `docker compose up -d postgres` and wait for the container health check to pass.
2. From `Backend/careerfit-backend`, run `mvnw.cmd spring-boot:run`. The host backend connects to Compose PostgreSQL at `localhost:5433`.
3. From `Frontend`, run `npm install` when dependencies are not present, followed by `npm run dev`. The local Vite application is available at `http://localhost:5173`.

Verification

1. Open `http://localhost:8080/actuator/health` and verify the expected aggregate, liveness, and readiness responses for the selected profile.
2. Open the frontend, verify public Job search, sign in with a designated test account, and confirm authenticated API access.
3. If OCR, mail, or another optional dependency is disabled, record the limitation instead of treating the missing external service as a verified production integration.

Shutdown and cleanup

1. Stop the frontend and backend processes, then use `docker compose down` to stop the local containers. Remove volumes only when test data loss is intended.

Appendix G. Role-Based User Guide

Guest

Open the Jobs page, search by title, skill, or company, select filters, and open a Job card to view its public details. Authentication is required before application or personalized scoring.

Candidate

Sign in, open My CVs, upload a PDF/DOCX/image file or save a manual draft. Review the extracted sections and quality warnings, correct them if needed, and confirm the CV before scoring. Then choose the default CV, review direct-match and Potential explanations, manage applications, use status and urgent filters, provide feedback, inspect recommendations, and configure AutoFit settings.

Recruiter

Sign in to the Recruiter workspace and complete the company profile before creating a Job. Save the Job as a draft, check JD quality, set the urgent flag and application deadline when needed, then publish it. Use Jobs and applicants for active recruitment, and use Talent Pool to filter Potential CVs, bookmark a Candidate, send an invitation, and update valid application states.

Administrator

Use the administrative workspace to inspect users, Jobs, audit records, notification and email-action state, and supported maintenance views. Suspend or reactivate only a designated test account and verify the resulting audit record.